# Keep Notes

A collaborative Google Keep-style notes application on the MERN stack — real-time sync, shared notes with roles, per-note version history, reminders, two-factor authentication and markdown notes. Built, tested and deployed end to end.

MongoDB 7 · Express 4 · React 18 · Node 20 · Mongoose 8 · Socket.IO · Vite 5 · Tailwind 3 · JWT + TOTP · Jest · Vitest · Playwright · Docker · GitHub Actions

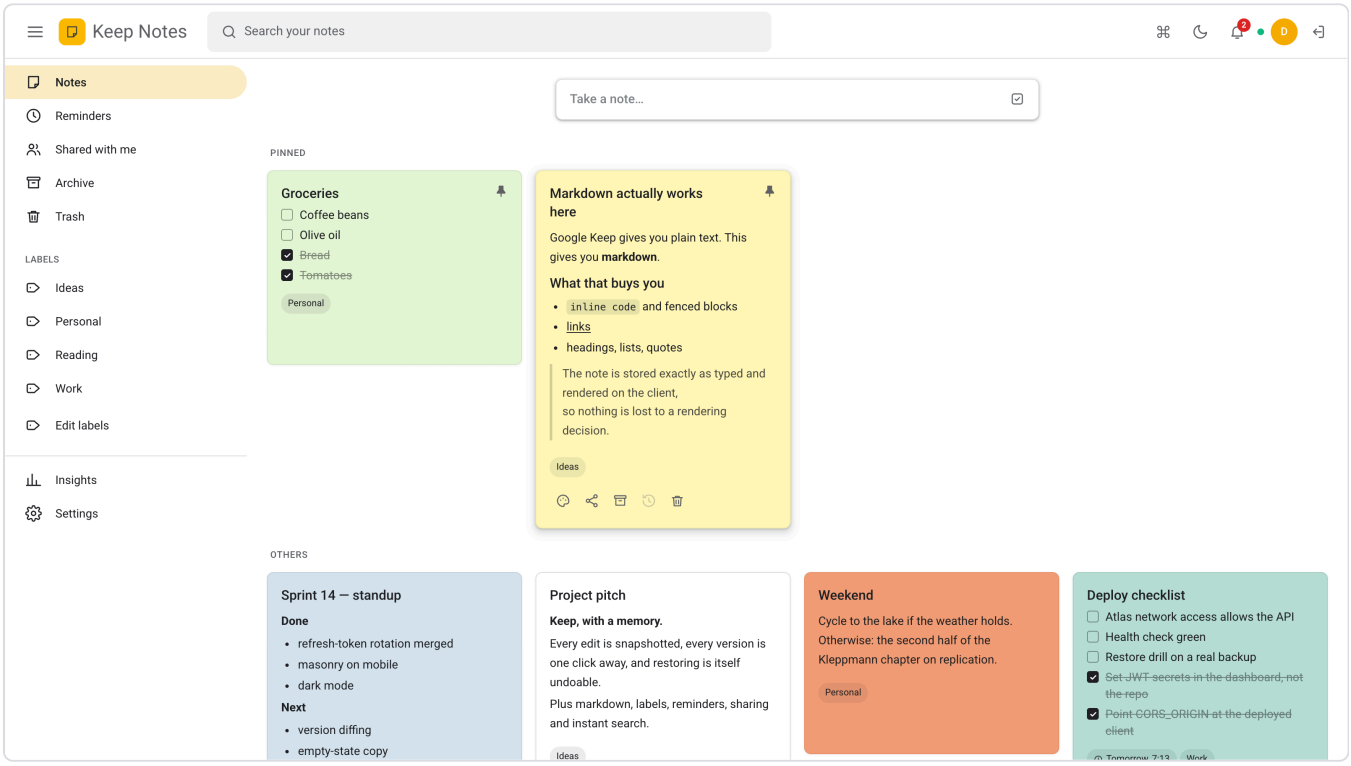

| | |
|---|---|
| Author | Muhammad Muttaqi |
| Context | Full-Stack Developer Intern, EyraTech Corporation — NASTP, Pakistan |
| Repository | github.com/muttaqi123/notes-app |
| Tests | 95 API · 24 client · a Playwright suite, all green in CI |


A written walkthrough of what was built, what it was built with, how it was built, and — for every decision an interviewer is likely to ask about — why it went the way it did.

# 00 What is in this document

Section 10 is the interview questions with answers, and section 11 is the bugs. Between them they are most of what anyone will ask about.

1. **The brief, and what was delivered** — the feature list, and where each item lives
2. **Tools and software** — every library and program used, and what each is for
3. **Architecture** — the one rule, the event bus, and the life of a request
4. **The data model** — seven collections, and why they are shaped that way
5. **Authentication** — two tokens, rotation, and two-factor
6. **Collaboration and real time** — roles, sockets, presence
7. **Version history and conflicts** — the feature Keep does not have
8. **Everything else** — reminders, attachments, export, activity, search
9. **The front end** — theming, the charts, and what was done about the bundle
10. **Testing, running, deploying** — what is covered and how it ships
11. **Interview questions, with answers** — the ones most likely to come up
12. **Bugs found and fixed** — the honest ones, with what each taught
13. **Limitations, and what comes next**




The board: pinned notes above the rest, masonry columns, colour-coded cards, checklists and markdown side by side, labels and views in the rail.

## 01 The brief, and what was delivered

The brief was to extend Google Keep with the things it does not do, while still delivering the Keep-style core end to end. That was version one. The app then grew a second half: collaboration.

### The four extensions the brief asked for

| REQUIRED | DELIVERED | WHERE |
| --- | --- | --- |
| **JWT authentication** | Registration and sign-in, 15-minute access tokens, opaque refresh tokens rotated on every use and stored only as hashes, per-device sessions you can revoke individually, TOTP two-factor with recovery codes, and password reset | `services/auth.service.js`, `twofactor.service.js` |
| **Markdown-formatted notes** | Notes written in markdown, a live preview toggle, rendered on cards, parsed with `marked` and sanitised with DOMPurify | `lib/markdown.js`, `NoteEditor.jsx` |
| **Tag-based filtering** | Labels as first-class documents, each one its own route, renameable everywhere at once | `services/labels.service.js` |
| **Note version history** | Every content edit snapshotted, a word-level diff against the current text, one-click restore — and the restore is itself undoable | `models/NoteVersion.js`, `VersionHistory.jsx` |

### The Keep-style core

Notes and checklists with full CRUD, eleven note colours in both light and dark, pin, archive, a soft-delete trash that empties itself after thirty days, an instant client-side search, and a responsive masonry board from one column on a phone to four on a wide screen.

## What was added beyond the brief

| FEATURE | WHAT IT DOES |
|---|---|
| **Sharing with roles** | A note can be shared as viewer or editor. Only the owner can share, change a role, or delete. |
| **Real-time sync** | An edit on one device appears on every other device that can see the note, and on every collaborator's board, without a refresh. |
| **Presence** | Avatars of everyone with the same note open. |
| **Conflict detection** | Two people saving at once: the second is shown both versions rather than silently overwriting the first. |
| **Reminders** | A time on a note; a sweep sends a notification when it comes due, to the owner and every collaborator. |
| **Image attachments** | Up to eight per note, re-encoded server-side before anything touches disk. |
| **Activity log** | Who changed what, and when — a different question from "what did it say", once a note has more than one person on it. |
| **Notification centre** | Reminders and share invitations, live over the socket. |
| **Command palette** | ⌘K to reach any note, view or action, plus a full keyboard-shortcut layer. |
| **Dark mode** | Light, dark, or follow the system — with Keep's own dark note palette. |
| **Insights** | A dashboard: counts, a thirty-day timeline, breakdowns by label and colour. |
| **Export** | Everything as JSON or as Markdown. |
| **Drag to reorder** | A manual sort, persisted, one request for the whole board. |

**WORTH SAYING OUT LOUD**

The starting point was a NestJS scaffold and an untouched Create React App — boilerplate with no logic in it. The brief asked for MERN, and Nest is not the E in MERN, so the honest move was to restructure rather than bolt Express onto a Nest app. Nothing of value was discarded; the scaffold had never had a line of business logic written into it.

# 02 Tools and software

Everything used to build, run, test and ship the app.

## The stack

| TOOL | VERSION | WHAT IT DOES HERE, AND WHY THIS ONE |
|---|---|---|
| **Node.js** | 20 LTS | The server runtime. ES modules throughout, so `import` works on both sides and there is no build step on the server. |
| **Express** | 4.21 | The HTTP layer. Small and explicit — the routing table *is* the framework, which keeps the interesting code out of it. |
| **MongoDB** | 7 | A note is naturally one document: a title, a body, an array of checklist items, references to labels. Storing it as one object is closer to what it is than three joined tables. |
| **Mongoose** | 8.9 | Schema, validation, indexes and `populate` — the four things the raw driver deliberately leaves to you. |
| **Socket.IO** | 4 | Real-time sync and presence. Chosen over a bare WebSocket for rooms, automatic reconnection, and the polling fallback for proxies that will not pass an upgrade. |
| **React** | 18.3 | Hooks only. State lives in one custom hook rather than a global store. |
| **Vite** | 5.4 | Dev server and bundler. Its proxy forwards `/api`, `/uploads` and `/socket.io` to Express, so there is no CORS in development. |
| **Tailwind CSS** | 3.4 | Utilities for layout and spacing; the theme itself is CSS custom properties that Tailwind reads (see §09). |

### Libraries doing one specific job

| LIBRARY | JOB |
|---|---|
| `jsonwebtoken` | Signs and verifies the access token |
| `bcryptjs` | Hashes passwords at cost 12 — deliberately slow, so guessing is slow |
| `otplib` + `qrcode` | TOTP codes and the enrolment QR |
| `zod` | Validates every request body at the edge, and generates the OpenAPI spec |
| `sharp` | Decodes and re-encodes every uploaded image — the security boundary, not a nicety |
| `multer` | Parses the upload, into memory rather than onto disk |
| `node-cron` | The reminder sweep and the nightly trash purge |
| `pino` + `pino-http` | Structured logs with a request id, credentials redacted |
| `helmet`, `express-rate-limit`, `cors`, `cookie-parser` | The HTTP hardening layer |
| `marked` + `dompurify` | Markdown to HTML, then HTML to *safe* HTML |
| `diff` | The word-level version diff |
| `recharts` | The insights charts, lazily loaded |
| `framer-motion`, `@dnd-kit` | Dialog transitions and drag-to-reorder |

### Development, testing and operations

| TOOL | WHAT IT WAS USED FOR |
|---|---|
| **Jest + supertest** | The API suite: the real Express app, driven in-process, no port |
| **mongodb-memory-server** | A real MongoDB in memory for the test run — real indexes, real validation, nothing to install. It doubles as the development fallback when `MONGODB_URI` is unset. |
| **Vitest + Testing Library** | The client suite: the sanitiser and the board's state machine |
| **Playwright** | End to end in a real browser, including sharing between two isolated browser contexts |
| **ESLint + Prettier** | Both packages lint clean at `--max-warnings 0` |
| **GitHub Actions** | Lint, both test suites and the build on every push; end-to-end after they pass |
| **Docker + Compose** | A local mongo:7, and a full three-service stack for anyone who would rather not install Node |
| **Puppeteer + headless Chrome** | Captured every screenshot in this document from the running app, and rendered this PDF |

CHOSEN AGAINST, ON PURPOSE

- **Redis** — nothing to cache and no cross-process state. Refresh tokens live on the user document, one query away regardless.
- **Redux / Zustand** — one screen owns the note state. A store would add indirection without removing any.
- **React Query** — a good answer to data fetching, and the reason one lint rule is switched off with a written explanation rather than obeyed. It would earn its place at the next size up.
- **TypeScript** — a real trade-off. The integration suite covers the API boundary, which is where shape errors actually bite; on a team I would add it.
- **A job queue** — two cron jobs. A queue would be infrastructure with nothing in it.

## 03 Architecture

THE ONE RULE EVERYTHING ELSE FOLLOWS

**Business logic lives in `server/src/services/`, and nothing in that folder imports Express.** A service takes plain values and returns plain values. It never sees a `Request`, never sets a status code, never builds a response.

### Why that rule survived contact with WebSockets

Real-time sync is exactly the kind of feature that usually breaks a layering rule, because the thing that needs to broadcast is buried in the business logic. It did not break this one.

```
notes.service.js          "this note changed, and these people can see it"
      │
      │  emitNoteChanged(note, audience, actorId)
      ▼
services/events.js        a plain Node EventEmitter. Knows nothing.
      │
      ▼
realtime/index.js         the ONLY file in the server that knows sockets exist
                          → io.to(`user:${id}`).emit('note:changed', …)
```

No service imports `realtime/`. The whole feature could be deleted without touching a line of business logic, and the test suite runs with nothing subscribed at all.

THE EVIDENCE, NOT THE CLAIM

Version two added sharing, roles, real-time sync, presence, reminders, attachments, an activity log and notifications — and **all 36 tests from version one passed untouched**. That is the only real way to know a layering rule is doing something.

### The life of one request

```
PATCH /api/notes/:id            "add a line to this note"
  │
  ├─ helmet, cors, json, cookies, pino-http      cross-cutting middleware
  ├─ requireAuth                    verifies the JWT → req.userId
  ├─ validate(updateNoteSchema)     zod parses the body; unknown keys stripped
  ├─ notesController.update         read the request, call a service, respond
  │    └─ notesService.updateNote(userId, noteId, patch, { expectedVersion })
  │         ├─ authorize(userId, noteId, WRITE)   ← one place, every time
  │         ├─ expectedVersion stale? → 409 carrying the current note
  │         ├─ does this patch touch content?
  │         ├─ if yes: snapshot the OLD state, then version += 1
  │         ├─ save
  │         ├─ emitNoteChanged(...)  → every device that can see it
  │         └─ log the activity
  └─ errorHandler                   the only place an error becomes a status
```

### Permissions, in one function

| LEVEL | WHO | MAY |
| --- | --- | --- |
| `read` | owner, editors, viewers | see the note, its history and its activity |
| `write` | owner, editors | edit the content, attach images |
| `own` | owner only | share, change roles, trash, delete |

Every note operation calls `authorize()`, so there is exactly one answer to "can this person do this" rather than a check repeated — and eventually forgotten — in each service function. **Every refusal is a 404, not a 403**, including a viewer trying to edit: a 403 confirms the note exists to someone being told it does not.

**WHY APP.JS AND INDEX.JS ARE SEPARATE FILES**

`app.js` builds the Express app and does nothing else — no port, no database, no sockets. `index.js` connects the database, attaches Socket.IO and listens. That split is what lets the test suite mount the entire API against an in-memory MongoDB and drive it in-process: no port to pick, no race between "the server is starting" and "the test is running", and no way for two test files to collide on a port.

## 04 The data model

Seven collections. Every one of them carries an owner.

```
users         { name, email(unique), passwordHash, avatarColor,
                settings: { theme, density, defaultView, sort },
                refreshTokens: [{ tokenHash, expiresAt, userAgent }],
                twoFactor:    { enabled, secret, backupCodes[] },
                passwordReset:{ tokenHash, expiresAt } }

labels        { owner, name, color }                unique (owner, name)

notes         { owner, title, body, type, items[], color,
                pinned, archived, trashedAt, labels[],
                attachments[], remindAt, reminderSentAt,
                order, version }
              index (owner, pinned desc, updatedAt desc)
              index (remindAt, reminderSentAt)        ← the sweep

noteversions  { note, owner, author, version, title, body, type,
                items[], color, labels[], reason }
              index (note, version desc)

noteshares    { note, owner, user, role }           unique (note, user)
                                                    index on user

activities    { note, actor, action, meta }         index (note, createdAt desc)

notifications { user, type, note, title, body, readAt }
```

### Six decisions in that schema

#### One document per note, not a notes table joined to an items table

A checklist item has no meaning apart from its note, is never queried on its own, and is always read and written with its parent. That is the exact case where embedding beats referencing — one read gets the whole note.

#### Labels are referenced, not embedded

The opposite call, for the opposite reason. A label is shared across many notes and has a life of its own: renaming it must change it everywhere at once. Embedded copies would need a fan-out write to every note that used it.

#### Shares are their own collection, indexed on the recipient

The query that matters most runs the other way from the obvious one: not "who can see this note" but "*which notes may this user see*". With an index on the recipient that is one lookup. As an array on the note it would be a scan of every note in the database.

#### Versions are their own collection too

They are written on every edit and read almost never. As an array on the note they would make every note document grow without limit — and that document is the hot path, since the board reads all of them at once. In their own collection the history costs nothing until someone opens it.

### `trashedAt` is a date, not a boolean

A boolean answers "is it in the trash". A timestamp also answers "since when", which is what the thirty-day auto-purge needs. It costs the same to store and answers a question a boolean cannot.

### `order` is a float

Dropping a card between two others is then one write — the new position is the midpoint of its neighbours — rather than renumbering everything after it.

> **THE INDEX THAT MATTERS**
>
> `{ owner: 1, pinned: -1, updatedAt: -1 }` is exactly the board query: this user's notes, pinned first, newest first. A compound index in the same order as the sort means MongoDB walks it and returns rows already ordered, instead of loading the set and sorting it in memory.

# 05 Authentication

Two tokens with two different jobs, and a third factor on top. This is the section interviewers push hardest on, so it is the one worth knowing cold.

| | ACCESS TOKEN | REFRESH TOKEN |
|---|---|---|
| **What it is** | A signed JWT | 48 random bytes — not a JWT |
| **Lifetime** | 15 minutes | 30 days |
| **Lives in** | A JavaScript variable, in memory | An `httpOnly` cookie |
| **Sent as** | `Authorization: Bearer …` | Automatically, by the browser |
| **Stored server-side** | Not at all — verified by signature | As a SHA-256 hash on the user document |
| **Revocable** | No, but it expires in 15 minutes | Yes, immediately |

### Why the access token is not in localStorage

`localStorage` is readable by any script running on the page. One XSS bug — a bad dependency, an unsanitised note — and the token is gone. Held in a module variable it dies with the tab and is never somewhere a script can enumerate. The cost is that a refresh loses it, which is precisely the job the refresh cookie does.

### Why the refresh token is not a JWT

Nothing about it needs reading without a database round trip, and a JWT can be forged by anyone who learns the signing secret. A random opaque string cannot be forged at all — it either matches a stored hash or it does not. Only the hash is stored, so a leaked database dump contains no usable token.

### Rotation

```
POST /api/auth/refresh
  ├─ hash the presented token, look up the user by that hash
  ├─ expired or unknown?  → 401, and delete the entry
  ├─ DELETE the presented token from the user      ← the rotation
  ├─ issue a NEW refresh token and store its hash
  └─ return a new access token, set the new cookie
```

A refresh token is good for exactly one use. If one is stolen and used, the real user's next refresh fails and they are signed out — the theft becomes visible instead of silent. Tokens are stored per device with the user agent, so the settings screen can list them and revoke one without touching the others.

### One subtlety in the client

Because rotation invalidates the old token, five requests firing on page load must not trigger five refreshes — four would be rejected and sign the user out. The API client keeps **one in-flight refresh promise** and lets every caller await it.

### Two-factor: why setup and confirmation are separate

```
POST /2fa/setup     generate a secret, return a QR code.  NOT enabled.
POST /2fa/confirm   user types a code from their app.     Now enabled,
                    and ten single-use recovery codes are shown once.
POST /2fa/disable   requires the password again.
```

Splitting them is the whole point. A user who scans the QR code and then loses their phone before confirming is *not* locked out of their own account, because the secret was never trusted. Enabling on setup would turn a half-finished enrolment into a lockout.

At sign-in, a correct password on a 2FA account returns **401 with the code `two_factor_required`** — a distinct code, so the client asks for a code rather than telling the user their password is wrong. Recovery codes are bcrypt-hashed and burned as they are used.

### The rest of the surface

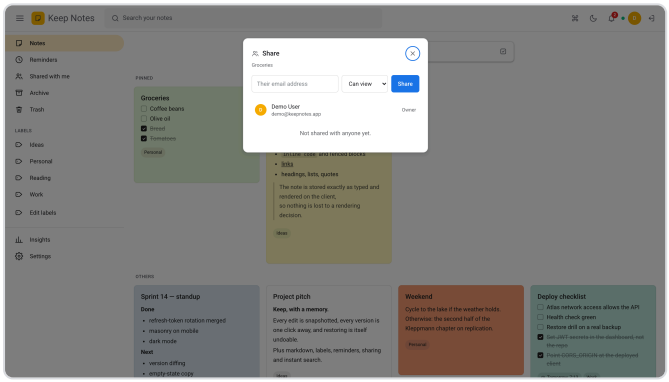
- **Identical failures.** A wrong password and an unknown email return the same message, and the code does the same work either way — returning early on an unknown email leaks it through timing. A test asserts the two messages match.
- **Password reset** answers identically whether or not the address is registered, so the endpoint is not a way to enumerate accounts. The token is hashed, expires in an hour, is single-use, and revokes every session when it is redeemed.
- **Changing a password revokes every session**, including the current one. If the reason for the change is that someone else had it, leaving their session alive defeats the point.
- **Secrets fail closed.** `config/env.js` refuses to start in production without real secrets rather than falling back to a development default — because a default shipped to production is a published private key.

# 06 Collaboration and real time

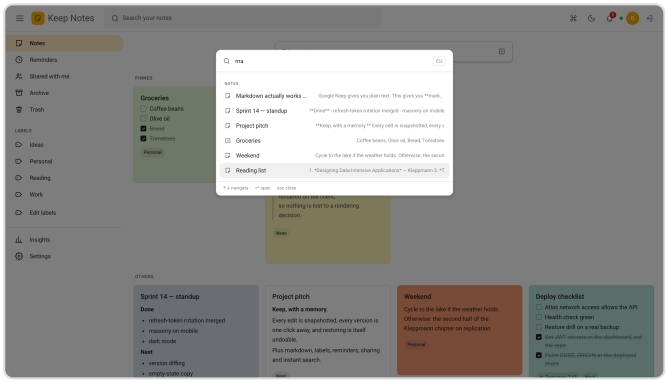
## Sharing

A note is shared by email address, as **viewer** or **editor**. Only the owner can share, change a role or revoke — an editor who could add more editors is a permission system that has quietly stopped being one.

- **Re-sharing changes the role** rather than failing or stacking a duplicate row.
- **Revoking emits a "removed" event** to that person specifically: to them the note has ceased to exist, so their board should drop the card without a refresh.
- **A collaborator can leave** a note themselves — the one share operation that is not the owner's to perform.
- **Label ids are filtered against the owner's own labels** before being attached, so nobody can attach a stranger's label by guessing an id.
- **Sharing with an unknown address says so.** This is not a login form; the address was typed by someone who already knows the person, and a silent no-op would be a broken feature.




Sharing, with roles. A viewer sees this panel read-only.




⌘K — notes, labels, views and commands in one list.

## The socket layer

Two kinds of room, and the distinction matters:

| ROOM | WHO IS IN IT | FOR |
|---|---|---|
| `user:<id>` | Every device signed in as that person | Note changes, removals, notifications |
| `note:<id>` | Everyone with that note open | Presence and typing only |

A change is delivered to the audience *the service computed* — the owner plus collaborators — so authorisation is decided once, in the service, and never re-derived in the socket layer. Joining a `note:` room runs the same `authorize()` the HTTP routes use, so presence cannot become a side channel telling you who is reading a note you cannot.

TWO DETAILS THAT ARE EASY TO GET WRONG

- **The token is read at connect time, not captured in a closure.** The access token rotates every fifteen minutes; a socket holding the token it was created with would silently fail to reconnect after the first rotation, and the app would look fine until someone noticed updates had quietly stopped.
- **A dropped connection has to leave every note it was viewing**, or the presence list slowly fills with ghosts. Presence is tracked per socket, not per user, so two tabs open and one closed does not read as "they left".

### Merging, not refetching

A change arrives as the note itself, so the board updates without a round trip and without losing the reader's scroll position. The client checks whether the note still belongs in the view on screen — a note archived on another device leaves the active board rather than sitting there stale.

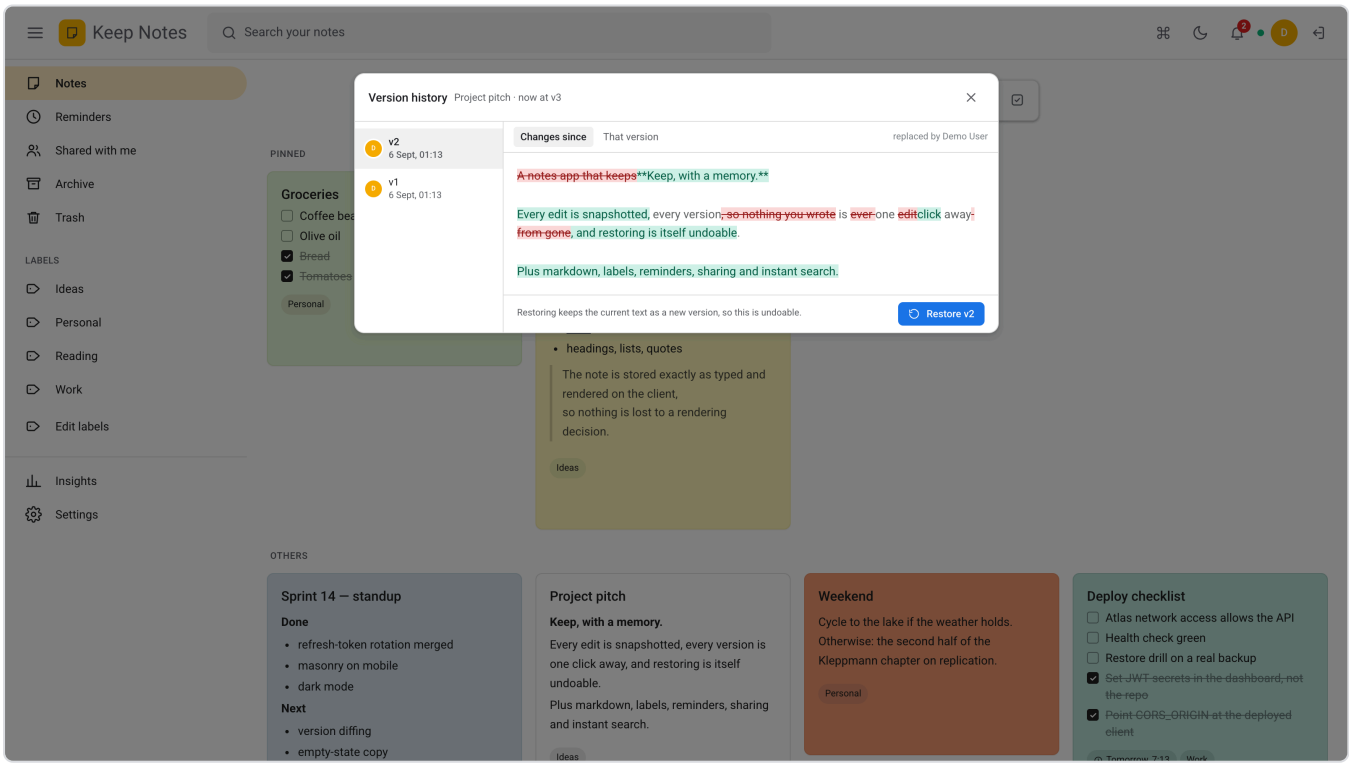
## 07 Version history and conflicts

Every note carries a `version`. On an edit that changes **content**, the server writes a snapshot of the state *before* the edit and increments the number.

| DECISION | REASONING |
|---|---|
| Snapshot the **old** state, before writing the new one | If the write of the new state fails, the history still describes reality. |
| Content only | Pinning, archiving or recolouring is not an edit to the text. A history full of "you pinned this" is a history nobody reads. |
| Compare before writing | A PATCH that sets the body to what it already was creates no version. Opening a note to read it must not add to its history. |
| Save on close, not on keystroke | One editing session becomes one version — a history a person can read. Per-keystroke versioning produces one nobody can. |
| Restoring is itself versioned | Restoring v1 snapshots the current text first, so going back is never how you lose the newer draft. |
| Each version records its **author** | Distinct from the owner once a note is shared: the history has to say which of them typed it. |
| Deleting forever deletes the history | No orphaned snapshots of a note that no longer exists. |

### The diff is word-level, not line-level

Notes are prose. A line diff of a reworded paragraph marks the whole paragraph changed and shows you nothing useful; a word diff shows the three words that actually moved.




Every past state on the left with its author, the change since then on the right, and a restore that says in the interface itself that it is undoable.

### Optimistic concurrency

A `PATCH` carries `expectedVersion`. If someone else saved in the meantime, the server refuses with **409** and attaches the note as it currently stands, and the editor shows both versions side by side with three choices: keep editing, discard mine, or save mine over it.

> **WHY THIS IS WORTH BUILDING**
>
> Last-write-wins is not an error the user ever sees, and that is exactly the problem: someone's paragraph disappears and nobody finds out until much later. The check is *opt-in* — only the editor sends the version, because a colour change does not need the protection and should not be blocked by an edit that happened in between. And nothing is lost either way: whichever version is not chosen is in the history.

## 08 Everything else

### Reminders

A time on a note; a sweep every minute sends what is due, to the owner *and* every collaborator — a shared to-do that only reminds its owner is not shared in any useful sense.

- **`reminderSentAt` is written after the notification exists**, so a crash mid-sweep retries rather than going silent. A missed reminder is the one failure this feature cannot have.
- **The query filters on `reminderSentAt: null`**, so a reminder goes out once however often the sweep runs or however many processes run it.
- **Changing the time clears it**, which re-arms a reminder that already fired.
- **An overlap guard** stops a second sweep starting while the first is still going — node-cron will happily do that, and two in flight would send some reminders twice.

### Attachments

Two things are deliberate, and both are security rather than polish:

1. **The file is re-encoded, never stored as uploaded.** Passing an untrusted file straight to disk and serving it back means trusting the uploader about what it is; a `.png` that is actually an HTML document is stored XSS the moment a browser sniffs it. Decoding with sharp and re-encoding to WebP means whatever comes out *is* an image, because an image encoder produced it — and it strips EXIF, which routinely carries GPS coordinates the uploader did not mean to publish.
2. **The stored name is a UUID, not the user's.** The uploaded filename is kept as a display label and never touches the filesystem, so a name like `../../etc/passwd` is a string, not a path.

Uploads are held in memory rather than written straight to disk, so a file that turns out not to be an image never becomes a file at all.

### Search — and why it is on the client

The API supports `?q=` and it is tested. The UI does not use it. Filtering the notes already in memory is instant, costs no request per keystroke, and works with a flaky connection. The server-side search stays for the day a user has more notes than one page holds — at which point the UI switches to it and nothing else changes.

The server version escapes regex metacharacters before building the query, so searching for `$40.00` searches for that text rather than executing it as a pattern. There is a test for exactly that.

### Pagination

Cursor-based, not `skip` / `limit`. `skip` has to walk and discard every row it skips, so page fifty costs fifty pages of work — and a note added while you are paging shifts every subsequent row by one, which duplicates or drops records under the reader. A cursor is a position in the sort: it costs the same on page fifty as on page one and cannot slip. One extra row is fetched per page purely to answer "is there another" without a second count query.

### The activity log

The version history answers *what did this note say*; the activity log answers *who changed it*. Once a note has more than one person on it those are different questions, and only one of them is answerable from snapshots.

## Export

JSON is the faithful copy — every field, restorable. Markdown is the readable one, which is what someone moving to another tool actually wants, and it works because the body was stored as markdown all along. Checklists come out as GitHub task lists so the checkboxes survive the move.

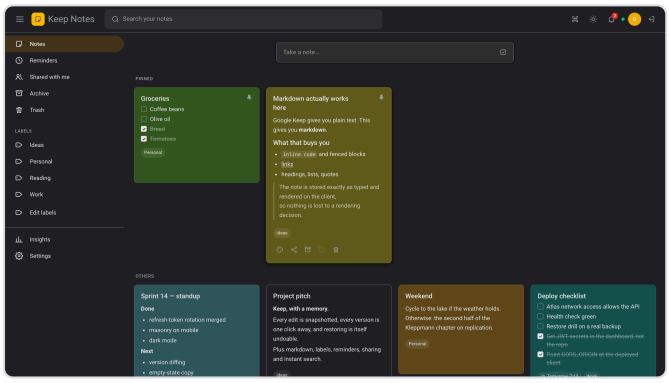
# 09 The front end

### Theming: why not Tailwind's `dark:` variant

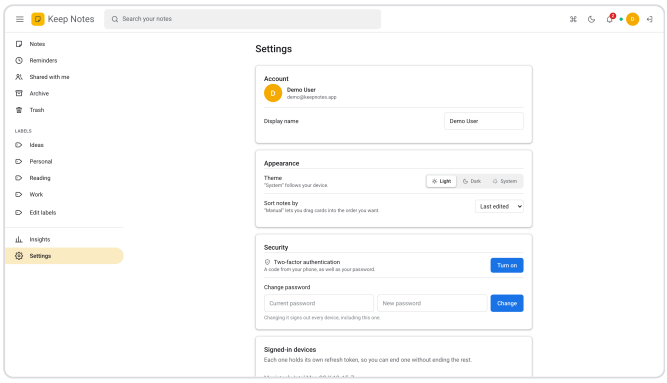
Eleven note colours in two themes is twenty-two values. A `dark:` variant on every element means writing each one twice and hoping nobody forgets one. Instead the theme is a set of **CSS custom properties** swapped by a `data-theme` attribute on `<html>`, and Tailwind reads them through `theme.extend.colors` — so `bg-surface` and `text-muted` are ordinary utilities that happen to be theme-aware, defined once.

The dark palette is Keep's own, not the light one dimmed: a tinted dark surface still reads as "the yellow note" without turning into a lamp.

> **THE WHITE FLASH**
>
> React cannot set the theme until it has mounted, and a dark-mode user seeing a white page for that moment is the one flash of unstyled content worth a synchronous script to avoid. A small blocking script in `index.html` reads the same `localStorage` key and stamps the attribute before the first paint. The choice is stored both there and on the account — locally so it is applied instantly, on the account so it follows the user to another device.




Dark mode, with Keep's own note palette rather than a dimmed light one.




Settings: theme, sort, two-factor, sessions, export.

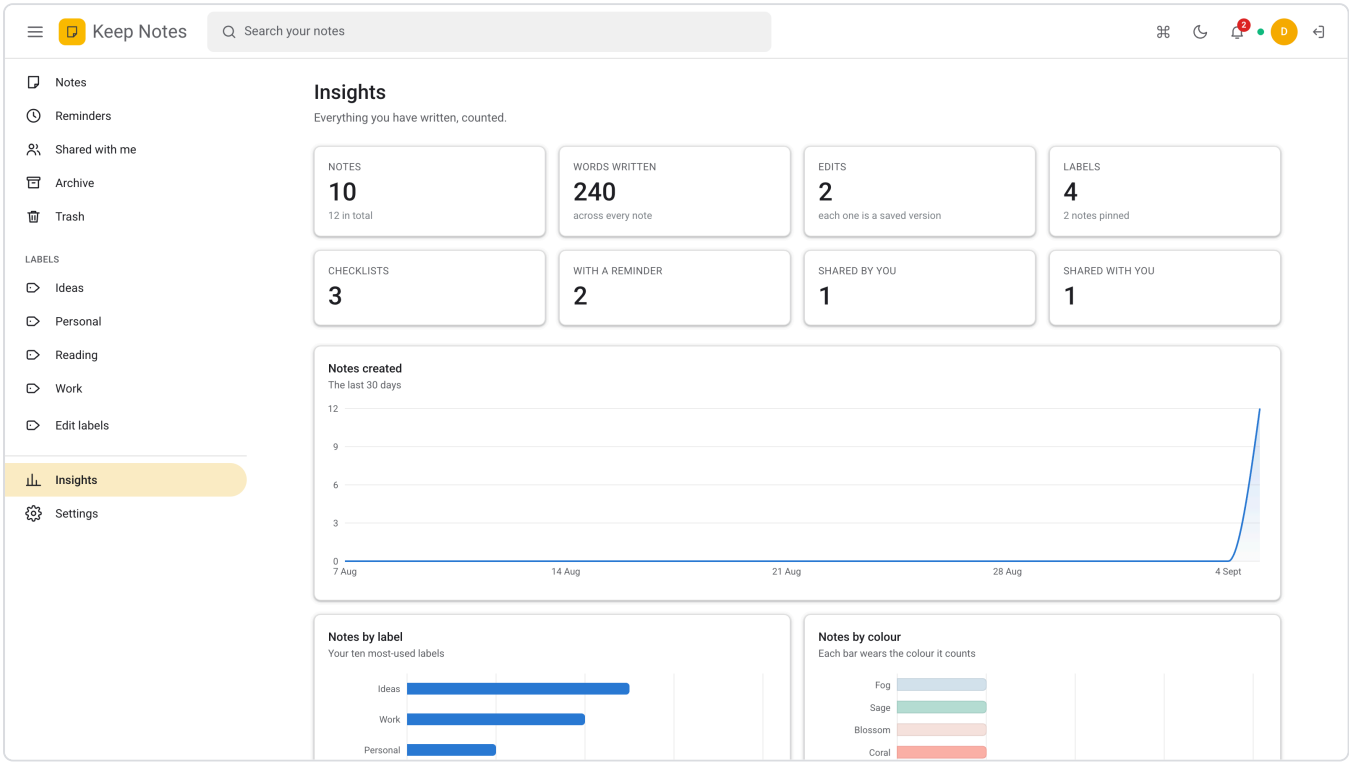
### The board

- **Masonry is CSS, not JavaScript.** Notes have wildly different heights; CSS multi-column with `break-inside: avoid` handles that natively — no resize observer, no measuring pass, no layout recalculation on every keystroke.
- **Dragging switches to a grid.** A column layout cannot express "this card is now between those two" while one is being moved, so the manual sort renders a grid instead. Dragging is only offered when the manual sort is chosen — dragging cards in a date-ordered list would be a lie about what happens next.
- **Infinite scroll** uses an `IntersectionObserver` on a sentinel below the board, which beats a button nobody wants to click.
- **Writes are optimistic**, and roll back with the error shown. A pin that silently un-pins itself is worse than one that says why.
- **A stale-response guard.** Switching views fires a request; a slow answer for a view you already left must not overwrite the one you are looking at. A request counter drops anything but the newest.

## The charts

Every chart on the insights screen is a **single series**, so none of them needs a legend or a categorical palette — the title says what the bars are. One accent hue, stepped separately for each theme rather than dimmed, and validated against both surfaces for lightness, chroma and 3:1 contrast rather than picked by eye.

The colour-breakdown chart is the one deliberate exception to "do not colour by category": there the colour *is* the category, so painting each bar the note's own colour is the honest encoding rather than a decorative one. Several of those colours sit below 3:1 against the card, so the same numbers are also offered as a table.




Insights: hero numbers first, then a thirty-day timeline and two ranked breakdowns.

## What was done about the bundle

The first build was 948 kB in one chunk. Splitting the vendor code by how often it changes and who needs it, and lazily importing the insights screen, brought the initial download to about 178 kB gzipped — the charting library is 112 kB of that and now only loads for the one screen that uses it.

```
react   53 kB gz   motion 43 kB gz   editor 26 kB gz   dnd 15 kB gz
app     42 kB gz   charts 112 kB gz  ← lazy, insights only
```

## Accessibility, briefly

- One `Modal` implementation, so Escape, click-outside, scroll lock and focus-into-panel are right everywhere rather than in some dialogs.
- Menus close on Escape as well as on an outside click — a menu you can only close with a mouse is a trap for anyone not using one.
- Single-letter shortcuts are ignored while typing in a field.
- A visible focus ring for keyboard users only, via `:focus-visible`.
- `prefers-reduced-motion` is honoured; someone who asked their system for less motion meant it.
- Toasts are announced with `role="status"` without stealing focus.

## 10 Testing, running, deploying

### What is covered

| SUITE | COUNT | COVERS |
|---|---|---|
| `auth`, `twofactor` | 28 | Registration, duplicate email, identical failure messages, the httpOnly cookie, refresh rotation, 2FA setup/confirm/disable, backup codes burning once, password reset, sessions |
| `notes`, `labels` | 19 | CRUD, markdown stored verbatim, checklists, pin/archive independence, sort order, trash and restore, search including a regex metacharacter, cross-account isolation |
| `sharing` | 13 | Roles, viewer cannot edit, editor cannot share or delete, re-share changes the role, revoke, leave, notifications |
| `concurrency` | 8 | Version conflicts, the 409 payload, no version written on a refused save, cursor pagination end to end |
| `versions`, `features` | 27 | History, restore-is-undoable, reminders and the sweep, the trash purge, activity, notifications, ordering, export, stats, settings |
| **Client** (Vitest) | 24 | The sanitiser against six XSS payloads; instant search; optimistic updates and rollback; live-update merging; label rename and delete |
| **End to end** (Playwright) | 11 | Sign-up, writing, search, trash round trip, version restore, sharing across two browser contexts, the palette, the theme toggle |

WHY THESE ARE INTEGRATION TESTS, NOT UNIT TESTS WITH MOCKS

The bugs worth catching here live in the seams — does the ownership filter actually filter, does the unique index actually reject, does a PATCH actually write a version. A mocked model cannot answer any of those. `mongodb-memory-server` gives real Mongo semantics with nothing to install, which is also why CI needs no database service.

### Continuous integration

Three jobs on every push: the API lints and tests, the client lints, tests and builds, and — only once both have passed, because a browser download is not worth spending on a broken commit — the end-to-end suite runs against a real API with its own in-memory database. A failed run uploads the Playwright report.

### Running it

```
npm run install:all
npm run dev            # API on :4000, client on :5173, docs at /api/docs

docker compose up -d mongo   # a database that persists
npm run seed                 # two accounts, so sharing can be demonstrated

docker compose up --build    # or the whole stack, on :8080
```

With `MONGODB_URI` unset the server starts an in-memory MongoDB on boot, so the app runs on a clean machine with nothing installed — which is exactly what you want when someone else is trying your project.

## Deploying it

| PIECE | WHERE | CONFIGURATION |
|---|---|---|
| Database | MongoDB Atlas | Free tier; the API's egress added to Network Access |
| API | Render, Fly, Railway | Root `server`, build `npm ci`, start `npm start`, health check `/api/health`; secrets in the dashboard |
| Client | Vercel or Netlify | Root `client`, build `npm run build`, output `dist`, `VITE_API_URL` set |

**THREE DEPLOYMENT DETAILS WORTH MENTIONING**

- **The SPA rewrite.** Without it, refreshing on `/archive` is a 404 from the CDN — it looks for a file, and the router never gets a chance to run.
- **A config file is not a guarantee.** `render.yaml` only applies to a service created *from* a Blueprint; one created by hand in the dashboard ignores it. That is why the secret check lives in `config/env.js` — a security default has to be safe in code, so that forgetting the deployment file is harmless.
- **Attachments are on disk**, so a host with an ephemeral filesystem needs a mounted volume — or S3, which is a change to one service file because nothing else knows where files live.

# 11 Interview questions, with answers

Shortest defensible answer first.

## On the stack

**Why MongoDB and not Postgres?**

A note is naturally one document — title, body, an array of items, references to labels — and it is always read whole. Embedding checklist items removes a join that would exist purely as a storage artefact. If the app grew reporting across users, or transactions spanning collections, that calculus would change.

**Why Mongoose over the raw driver?**

Schema, validation, index declaration and `populate`. The driver leaves all four to you, and hand-rolled versions of them are exactly where inconsistent data creeps in.

**Why Socket.IO rather than a plain WebSocket?**

Rooms, automatic reconnection with backoff, and a polling fallback for proxies that will not pass an upgrade. I would use a raw WebSocket if I needed the smallest possible payload; here the operational behaviour is worth more than the bytes.

**Why no TypeScript?**

A real trade-off, and I would add it on a team. The integration suite covers the API boundary, which is where shape errors actually bite. On a codebase this size the tests were the higher-value spend.

## On authentication

**Walk me through your JWT flow.**

Sign-in returns a 15-minute access JWT in the body and sets an `httpOnly` refresh cookie. The client keeps the access token in memory and sends it as a Bearer header. On a 401 it refreshes once and retries transparently. Refresh rotates: the presented token is deleted as it is used and a new one issued.

**Where do you store the token, and why not localStorage?**

In memory. `localStorage` is readable by any script on the page, so a single XSS bug is a stolen session. In memory the token dies with the tab, and the `httpOnly` cookie — which JavaScript cannot read at all — is what survives a reload.

**A JWT can’t be revoked. How do you sign someone out?**

You revoke the refresh token, which is server-side state, and the access token expires within fifteen minutes. That is the trade: a short window of un-revocable access in exchange for stateless request verification. It is also why the settings screen can list devices and end one of them.

**Why rotate refresh tokens?**

So a stolen one is good for a single use, and so the theft becomes visible — the real user’s next refresh fails and signs them out, instead of an attacker holding a valid 30-day session silently.

**Why is two-factor setup two steps?**

So that scanning the QR code enables nothing. If confirmation and enabling were the same step, a user who scanned and then lost their phone would be locked out of their own account. The secret is stored but not trusted until they prove they can read a code from it.

## On collaboration

**How do you stop one user reading another's note?**

Every note operation goes through one `authorize()` function that resolves the caller's role — owner, editor, viewer or none — and compares it to the level the operation needs. It returns 404 rather than 403, because a 403 confirms the id exists. Thirteen tests cover the matrix.

**How does real-time sync not break your layering rule?**

A service announces "this note changed, and these people can see it" on an in-process EventEmitter. The Socket.IO layer subscribes and forwards. No service imports it, so it could be deleted without touching business logic — and the test suite runs with nothing subscribed. The proof is that all 36 tests from version one passed untouched through the rewrite.

**What happens when two people edit the same note?**

The client sends the version it believes it is editing. If it is stale the server answers 409 with the note as it now stands, and the editor shows both versions and asks. Nothing is lost either way, because whichever is not chosen stays in the history.

**Why not operational transforms or CRDTs, like Google Docs?**

Because the editing model is different. Keep-style notes are written by one person at a time and occasionally by two; character-level merging is a large amount of machinery for a case that arises rarely, and it is genuinely hard to get right. Detecting the conflict and asking is honest, cheap, and never loses text. If simultaneous character-level editing became the main use, a CRDT would be the right rewrite.

**How do you know a socket client is who it says?**

The handshake carries the same access token as an HTTP request and is verified in middleware before any room is joined, so an unauthenticated connection hears nothing. Joining a note's presence room runs the same authorisation the REST routes use.

## On the features

**Why a separate versions collection and not an array on the note?**

Versions are written often and read almost never. An array would make every note document grow without bound, and that document is on the hot path — the board loads all of a user's notes at once. In its own collection the history costs nothing until someone opens it.

**Isn't storing full snapshots wasteful? Why not diffs?**

Yes, and deliberately. Notes are small, restoring a snapshot is a copy while restoring a diff chain is a replay, and a corrupt snapshot loses one version where a corrupt diff loses everything after it. Diffs would be right for large documents; these are not. The diff is computed at read time for display, which is where it is actually wanted.

**You render user content as HTML. What about XSS?**

Markdown is parsed with `marked`, then the HTML goes through DOMPurify with an explicit tag and attribute allow-list and a URI pattern that blocks `javascript:`. Six client tests fire real payloads at it. Uploaded images are re-encoded through sharp, so a file claiming to be a PNG but containing HTML cannot be served back as HTML.

**Why is search client-side?**

It is instant, fires no request per keystroke, and works offline. The server-side search exists and is tested, for when a user has more notes than one page holds — switching to it is a change in one hook.

**Why cursor pagination rather than skip and limit?**

`skip` walks and discards every row it skips, so deep pages get linearly slower, and a row inserted while you page shifts everything after it — which duplicates or drops records. A cursor is a position in the sort: constant cost, and stable under concurrent writes.

## On the engineering

**How is the code organised?**

Business logic in `services/`, which imports nothing from Express. Controllers read the request, call a service and respond. Permissions in one file. The socket layer in one file that nothing imports. That is what makes the logic directly testable and what let version two be an addition rather than a rewrite.

**What does an optimistic update roll back?**

The whole previous notes array, captured before the local change from a ref that mirrors state. On failure it is restored and the error surfaced. It has to come from a ref rather than a setState updater — that was a real bug, and it is in section 12.

**Tell me about a bug you hit.**

Section 12 is four of them. The one I would lead with is the optimistic rollback that never ran: the previous state was captured inside a `setState` updater, which React runs during the render pass rather than at the call site, so by the time a rejected request reached its catch the variable was often unassigned — and the rollback silently did nothing. A client test caught it.

**How would this scale?**

The board query is a single indexed read per user and MongoDB shards naturally on `owner`. The API is stateless apart from refresh tokens in the database, so it scales horizontally — except the socket layer, which holds presence in process memory and would need a Redis adapter to run on more than one instance. That is the first thing I would change.

**What is the weakest part?**

Presence in process memory, as above. After that, attachments on local disk rather than object storage, and the fact that the reminder sweep would double-send if two instances ran it — the row-level guard makes that a small window rather than a systematic problem, but it is a window.

## 12 Bugs found and fixed

Each of these was found by a test, a linter or a screenshot rather than by reading the code, which is the argument for having all three.

### 1. The rate limiter that allowed nothing

**Symptom:** 35 of 36 tests failed with `429 Too Many Requests` on the very first run.
**Cause:** `limit: isTest ? 0 : 30`. In express-rate-limit v7 a limit of zero means *allow zero requests*, not *disabled*.
**Fix:** `skip: () => isTest`.
**Lesson:** I had assumed an API's semantics instead of checking them. The symptom named the cause immediately, which is what a good test suite buys.

### 2. The optimistic rollback that never ran

**Symptom:** two client tests asserting that a failed update restores the previous state — both failed.
**Cause:** the previous state was captured *inside* a `setState` updater. React runs updaters during the render pass, not at the call site, so when a rejected promise reached its `catch` the variable had often never been assigned — and `if (previous)` quietly skipped the rollback.
**Fix:** a ref that mirrors the state, written after each commit and read synchronously by the handlers.
**Lesson:** this was invisible in manual testing, because it only shows up when a request fails. It is the exact class of bug tests exist for.

### 3. The chart that lost a day

**Symptom:** a stats test passed in the morning and failed in the evening.
**Cause:** the thirty-day timeline built its buckets from local midnight while MongoDB's `$dateToString` grouped in UTC. East of Greenwich the two disagree for part of every day, so the last bucket came up empty.
**Fix:** bucket in UTC on both sides.
**Lesson:** a test that fails depending on the clock is not flaky — it is telling you there is a timezone assumption you did not write down.

### 4. The checkbox that painted nothing

**Symptom:** in a screenshot, ticked checklist items had no filled box.
**Cause:** `bg-current` on an element that also carried `text-transparent`. `currentColor` *is* the property being set to transparent, so the fill resolved to nothing.
**Fix:** state the box colour and the tick colour outright instead of routing both through `currentColor`.
**Lesson:** worth looking at the rendered thing. No test would have caught this, and it was invisible in the code.

### 5. The first screen that stayed invisible

**Symptom:** the login card rendered at 46 % opacity in an automated browser.
**Cause:** an entrance animation whose *resting* state is `opacity: 0` until JavaScript finishes it. In a tab the browser considers hidden, animations never advance.
**Fix:** no entrance animation on the first screen. Animation is for things that appear after a click, where the tab is certainly awake.
**Lesson:** an element that is invisible until an animation runs has made its visibility depend on something that can fail.

**AND ONE NON-BUG WORTH KNOWING**

The strict React Compiler lint rules flagged eleven things. Ten were real and were fixed — refs written during render, state derived by effect rather than computed, a variable mutated while rendering a list. One was `set-state-in-effect` objecting to fetching data in an effect, which is a fair objection with a real answer (a data-fetching library) that this project deliberately does not use. That rule is switched off in the config, with the reasoning written next to it. The distinction — fix ten, document one — is the point.

## 13 Limitations, and what comes next

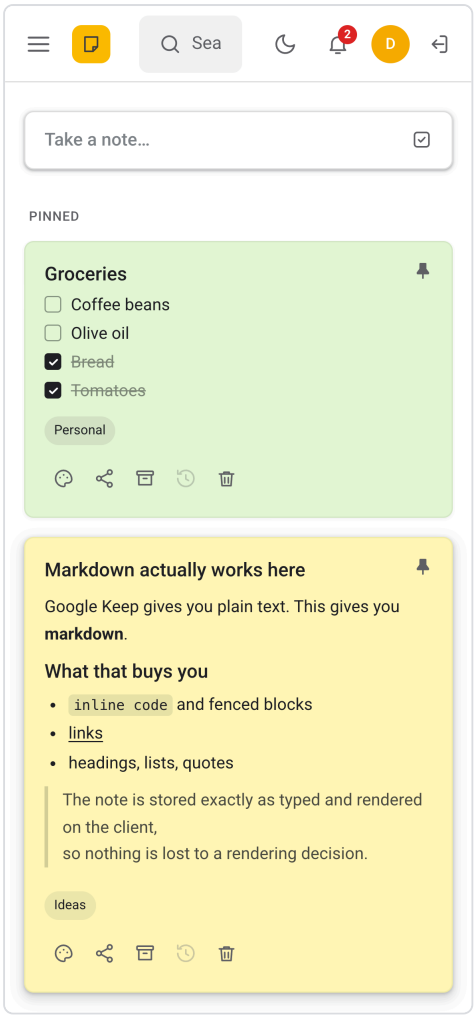
Knowing what is missing reads better than claiming nothing is.

### Known limitations

- **Presence lives in process memory.** Two API instances would each know only their own viewers. The fix is the Socket.IO Redis adapter, and it is the first thing I would change before scaling out.
- **The reminder sweep is not distributed.** The `reminderSentAt` guard makes a double-send a narrow race rather than routine, but two instances running the cron would occasionally both send.
- **Attachments are on local disk.** Fine behind one instance with a volume; object storage is the real answer, and it is a change to one service file.
- **No character-level collaborative editing.** Conflicts are detected and surfaced, not merged. Deliberate — see the CRDT answer in section 11.
- **No email.** Password reset and share invitations are real flows with real tokens; the delivery step logs instead of sending. Wiring a provider is one function.
- **The stats timeline is bucketed in UTC**, not the reader's timezone, because the API does not take one. It should.
- **No offline support.** The client is online-only; a service worker and a queued mutation log would be a project of its own.

### What I would build next, in order

1. **The Redis adapter for Socket.IO**, and a distributed lock for the sweep — the two things standing between this and more than one instance.
2. **Object storage for attachments**, behind the same service interface.
3. **A timezone on the stats request**, so the timeline means what the reader thinks it means.
4. **Email delivery**, which turns two flows from demonstrable into usable.
5. **TypeScript**, if this were going to be worked on by more than one person.




390 px: one column, the nav rail slid away, actions always visible because there is no hover on a touch screen.

**IF YOU REMEMBER ONE THING FROM THIS DOCUMENT**

The services rule is the spine of the project: every rule about a note lives in one framework-free folder, permissions have exactly one implementation, and the real-time layer is a subscriber that nothing imports. That is what made version two an addition rather than a rewrite — and the evidence is that all thirty-six of version one's tests passed untouched. Every other decision here is a trade-off with a reason attached, and the reason is the part worth saying out loud.